

DRY-RUN GUIDE | 02_armstrong_number.java
Source: java\2nd_belt\02_armstrong_number.java

CORE BEHAVIOR

Counts digits, sums each digit raised to that count using repeated multiplication, then compares to original. For input 0, digit count remains 0 but the empty sum is zero, so it reports Armstrong.

REPRESENTATIVE INPUT / CALL

Input: '153'.

STEP TRACE

digits=3; sum=1^3+5^3+3^3=1+125+27=153.

EXPECTED OUTPUT / RETURN

'Armstrong'

ACTUAL SOURCE EXCERPT

Verbatim lines from the source; long lines wrap. Source files were not modified.

```
001 | /*
002 | Question:
003 | A number is an Armstrong number if the sum of each digit raised to the
004 | power of the number of digits equals the original number.
005 |
006 | Example:
007 | 153 = 1^3 + 5^3 + 3^3 = 153
008 |
009 | Input:
010 | A single integer N.
011 |
012 | Output:
013 | Print "Armstrong" if the number is Armstrong, otherwise print "Not Armstrong".
014 | */
015 |
016 | import java.util.Scanner;
017 |
018 | public class ArmstrongNumber {
019 |     public static void main(String[] args) {
020 |         Scanner sc = new Scanner(System.in);
021 |         int n = sc.nextInt();
022 |
023 |         int original = n;
024 |         int digits = 0;
025 |         int temp = n;
026 |
027 |         while (temp != 0) {
028 |             digits++;
029 |             temp /= 10;
030 |         }
031 |
032 |         int sum = 0;
033 |         temp = n;
034 |
035 |         while (temp != 0) {
036 |             int digit = temp % 10;
```

```
037 |         int power = 1;
038 |
039 |         for (int i = 0; i < digits; i++) {
040 |             power *= digit;
041 |         }
042 |
043 |         sum += power;
044 |         temp /= 10;
045 |     }
046 |
047 |     System.out.println(sum == original ? "Armstrong" : "Not Armstrong");
048 |     sc.close();
049 | }
050 | }
051 |
```